ASSIGNMENT 2 – TASK 5

SYNCHRONISATION: CRITICAL SECTIONS, SEMAPHORES AND MONITORS

1. Introduction and Problem Context

The assignment presents a railway ticket-booking system in which thousands of agents may attempt to reserve seats on the same trains at the same time. Without synchronisation, two agents may sell the same seat. At the other extreme, one global lock can serialize the entire booking system and reduce throughput. The design must protect shared state while allowing independent bookings to proceed concurrently.

Task 5 requires evaluation of the critical-section requirements, Peterson's two-process solution, binary and counting semaphores, bounded-buffer, readers-writers and dining-philosophers problems, monitors with condition variables, bounded-buffer testing, lock-granularity throughput measurement and an alternative shared-counter design using C11 atomics or a spin lock.

2. Critical-Section Problem

A critical section is the part of a program that accesses or updates shared data. For railway booking, checking whether a seat is free and marking it booked must be treated as one indivisible transaction with respect to competing agents.

Requirement	Meaning	Railway booking application
Mutual exclusion	At most one process/thread at a time executes the protected critical section for the same shared resource.	Two agents cannot simultaneously confirm the same seat.
Progress	If no thread is in the critical section and some want to enter, selection of the next entrant cannot be postponed indefinitely by unrelated threads.	A free seat must be available to a waiting booking agent without unnecessary delay.
Bounded waiting	After a thread requests entry, there is a finite bound on the number of other entries before it is allowed in.	An agent must not be postponed forever by repeated bookings from other agents.

A correct solution must preserve all three properties. Mutual exclusion alone is insufficient if a thread can starve or if the system can remain idle while a thread is waiting.

3. Peterson's Solution for Two Processes

Peterson's algorithm is a software mutual-exclusion solution for two participants, conventionally numbered 0 and 1. Each participant has a flag indicating interest in entering the critical section. The shared turn variable gives the other participant priority when both compete.

For participant i, let j = 1 − i. Entry protocol: flag[i] = true; turn = j; while (flag[j] && turn == j) wait. On leaving, flag[i] = false.

Why the classical algorithm satisfies the three requirements

Property	Reasoning
Mutual exclusion	If both flags are true, the shared turn value can favour only one participant. Both cannot simultaneously satisfy the condition to pass the waiting loop.
Progress	When one participant is not interested, the other does not wait for it. When both are interested, turn resolves the competition.
Bounded waiting	A participant gives the other the turn before waiting. Once the other has entered and later clears its flag, the waiting participant can proceed; repeated overtaking is bounded for two participants.

Modern processor and compiler limitation

The textbook proof assumes atomic, sequentially consistent reads and writes with the required visibility order. Modern processors and compilers may reorder or delay ordinary memory operations. Cache coherence alone does not make the textbook sequence a complete memory-ordering protocol. Declaring the flags volatile is not enough: volatile does not provide inter-thread atomicity or a happens-before relation in C. A correct C implementation must use atomic variables and suitable memory ordering, or use a platform-supported lock primitive. The supplied peterson_demo.c uses C11 atomic variables with memory_order_seq_cst to make the ordering explicit.

The Peterson demonstration starts two threads. Each enters the protected region 200,000 times to increment a shared counter. The expected total is 400,000. This is a demonstration of the two-participant algorithm with C11 atomics, not a recommendation to replace a production pthread mutex with Peterson's algorithm.

4. Binary and Counting Semaphores

A semaphore is a synchronisation variable accessed through atomic wait and signal (often called P/V or down/up) operations. wait decreases the semaphore or blocks until a resource is available. signal increases it and may wake a blocked waiter.

Semaphore type	Meaning	Example use
Binary semaphore	Value is used as 0/1 availability; can provide mutual exclusion when used with a disciplined protocol.	Protect one shared seat-record update or one single-instance device.
Counting semaphore	Value represents the number of available identical resources or slots.	Track a pool of available seats, buffer slots or permits.

For a seat inventory, a counting semaphore can represent the number of seats still available, while a mutex protects the compound operation that selects a particular seat and updates its state. The semaphore count alone does not identify which seat is free. Booking should wait for availability, lock the relevant seat/inventory data, select and mark one seat, update the count consistently, then release the lock. A cancellation returns a permit after the seat is safely restored to the available state.

5. Bounded-Buffer (Producer–Consumer) Problem

A bounded buffer has a fixed number of slots. A producer must wait when the buffer is full, and a consumer must wait when it is empty. The standard semaphore solution uses empty = buffer

capacity, full = 0 and a mutex to protect the shared indices and buffer operations.

Producer sequence

```
wait(empty);                 // reserve an empty slot
lock(mutex);                // protect buffer and in index
buffer[in] = item;
in = (in + 1) % SIZE;
unlock(mutex);
signal(full);               // advertise one filled slot
```

Consumer sequence

```
wait(full);                  // wait for an available item
lock(mutex);                // protect buffer and out index
item = buffer[out];
out = (out + 1) % SIZE;
unlock(mutex);
signal(empty);              // advertise one free slot
record_consumed_item(item);
```

In the completed C program, the sum and per-item occurrence counters are maintained atomically after removal. The buffer contents and circular indices are protected by the mutex. The program supports the original one-producer/one-consumer run and a four-producer/four-consumer run. Each producer creates 20 unique item IDs. For 1P/1C, the values 1 through 20 sum to 210. For 4P/4C, the values 1 through 80 sum to 3,240. The program checks every expected item was consumed exactly once, not merely that the total sum matches.

5.1 Required test results

Configuration	Items produced/consumed	Expected sum	Validation
1 producer / 1 consumer	20	210	Every item exactly once
4 producers / 4 consumers	80	3,240	Every item exactly once
Original test repeated 100 times	20 per run	210 on each run	100/100 runs must pass

The run script executes the 1P/1C and 4P/4C tests, then repeats the original 1P/1C case 100 times. It records each run and confirms that the sum is 210 and each item is accounted for exactly once on every run.

5.2 Removing the mutex

The empty and full semaphores count slots, but do not provide mutual exclusion between multiple producers updating the shared in index or multiple consumers updating the out index. Without the mutex, two producers may read the same in value and overwrite the same slot, or two consumers may read the same out value. The program therefore includes an optional UNSAFE_NO_MUTEX build for observation. In C, such unsynchronised concurrent access to non-atomic shared indices is a data race and undefined behaviour. The unsafe output can appear correct on one run by chance; that does not establish correctness. The mutex must remain in the final implementation for the 4P/4C case.

6. Readers–Writers Problem

The readers–writers problem concerns a shared data set that may be read by several readers concurrently, but a writer needs exclusive access. A classic reader-preference semaphore solution uses a mutex to protect read_count and a writer semaphore to exclude writers while at least one reader is active.

```
semaphore mutex = 1, wrt = 1;
int read_count = 0;

Reader:
  wait(mutex);
  read_count++;
  if (read_count == 1) wait(wrt);
  signal(mutex);
  perform_read();
  wait(mutex);
  read_count--;
  if (read_count == 0) signal(wrt);
  signal(mutex);

Writer:
  wait(wrt);
  perform_write();
  signal(wrt);
```

This version permits multiple readers together and excludes writers during reading or writing. However, continuous arrival of readers can starve a writer. A fair or writer-preference queueing policy is needed when bounded waiting for writers is required.

7. Dining-Philosophers Problem

Five philosophers alternate between thinking and eating. Each needs the two adjacent forks to eat. If every philosopher picks up one fork and waits for the other, a circular wait can occur and the system may deadlock.

A semaphore solution can use one binary semaphore per fork and a counting semaphore room initialized to N−1. A philosopher waits on room before picking up the two forks, takes the forks, eats, releases both forks and then signals room. Allowing at most N−1 philosophers to compete for forks breaks the condition needed for circular wait and prevents this deadlock pattern. A fair semaphore policy is still desirable to reduce starvation.

```
semaphore room = N - 1;
semaphore fork[N] = {1, 1, ..., 1};

Philosopher i:
  wait(room);
  wait(fork[i]);
  wait(fork[(i + 1) % N]);
  eat();
  signal(fork[(i + 1) % N]);
  signal(fork[i]);
  signal(room);
```

Another valid deadlock-avoidance technique is to impose a global ordering on forks and always acquire the lower-numbered fork first. A consistent resource order prevents a circular-wait cycle.

8. Monitors and Condition Variables

A monitor encapsulates shared data and the operations that may access it. Only one thread executes a monitor procedure at a time, so mutual exclusion is built into the abstraction. Condition variables

let a thread sleep until a state predicate becomes true. A waiting thread releases the monitor lock while it sleeps and reacquires it before continuing.

```
Monitor SeatInventory:
  int available;
  condition seat_available;

  procedure book_seat():
    lock(monitor)
    while available == 0:
      wait(seat_available, monitor_lock)
    available = available - 1
    unlock(monitor)

  procedure cancel_seat():
    lock(monitor)
    available = available + 1
    signal(seat_available)
    unlock(monitor)
```

The predicate is checked in a while loop rather than an if statement because condition waits can wake spuriously and another thread may change the state before the awakened thread reacquires the lock. Monitors make the data, lock discipline and wait/signal conditions easier to reason about as one unit.

9. Railway Booking Lock-Granularity Decision

Three designs are compared in booking_locks.c. The benchmark simulates 16 agents, each making 10,000 booking attempts, across 8 trains, 8 coaches per train and 512 seats per coach. Each strategy runs three trials and reports the median elapsed time, requests per second and successful bookings. These are controlled simulation parameters, not real railway inventory.

Design	Concurrency behaviour	Trade-off
One global lock	Every booking operation uses one mutex.	Simplest to reason about, but all trains and coaches serialize.
One lock per train	Bookings on different trains can proceed concurrently.	More concurrency than a global lock; requests on the same train still serialize.
One lock per coach	Independent coaches can be booked concurrently.	Finest granularity in this simulation; more locks to manage and multi-coach operations need a consistent lock order.

Record the measured table from booking_lock_results.txt:

Lock strategy	Median time (ms)	Requests per second	Successful bookings
One global lock	Local benchmark output	Local benchmark output	Local benchmark output
One lock per train	Local benchmark output	Local benchmark output	Local benchmark output
One lock per coach	Local benchmark output	Local benchmark output	Local benchmark output

The likely design direction is finer-grained locking when transactions touch only one coach, because unrelated coaches can proceed independently. The measured throughput must decide the final recommendation. If a booking spans several coaches or trains, acquire locks in a fixed global order and release them reliably to avoid deadlock.

10. Alternative Concept: C11 Atomics and Spin Locks

For a simple shared scalar counter, C11 atomic_fetch_add can perform an indivisible increment without a separate semaphore/mutex protocol. It is normally the best fit when the entire critical operation is exactly one atomic update. It does not automatically protect a multi-step transaction that reads a seat, checks availability, updates multiple fields and commits a reservation.

A spin lock repeatedly tests a flag until it becomes available. It can have low acquisition overhead when the critical section is extremely short and contention is low, but waiting threads consume CPU while spinning. Under heavy contention or a long critical section, spin time wastes processor capacity. A pthread mutex can block waiting threads, making it more suitable for longer critical sections or oversubscribed workloads.

The atomic_spin_benchmark.c program compares C11 atomic_fetch_add, an atomic_flag spin lock and a pthread mutex for 1, 4 and 16 threads. Each thread performs 200,000 increments; each configuration is repeated three times. The program reports median wall time, nanoseconds per update, aggregate worker CPU time and failed spin attempts.

Threads	Method	Median ms	ns/update	Aggregate worker CPU ms	Failed spins
1	atomic_fetch_add	Record output	Record output	Record output	0
1	spin lock	Record output	Record output	Record output	Record output
1	pthread mutex	Record output	Record output	Record output	0
4	atomic_fetch_add	Record output	Record output	Record output	0
4	spin lock	Record output	Record output	Record output	Record output
4	pthread mutex	Record output	Record output	Record output	0
16	atomic_fetch_add	Record output	Record output	Record output	0
16	spin lock	Record output	Record output	Record output	Record output
16	pthread mutex	Record output	Record output	Record output	0

Aggregate worker CPU time is the sum of thread CPU times and can exceed wall-clock time when threads run on multiple cores. Failed spin attempts indicate how often threads found the lock busy; they are a useful indicator of spinning, not a direct count of CPU cycles. Use the measured results from the local machine to explain how contention changes from 1 to 4 to 16 threads.

For a long critical section, use a blocking mutex or another blocking synchronisation primitive rather than a spin lock. Use atomic_fetch_add for a simple counter when one atomic operation is

sufficient. A spin lock is considered only for very short critical sections when the expected wait is shorter than the cost of blocking and the system has suitable CPU capacity.

11. Complexity and Correctness Considerations

Bounded-buffer producer and consumer operations perform O(1) work per item, excluding time blocked while waiting for a semaphore. For M total items, total data movement and accounting are O(M); buffer storage is O(SIZE). The lock-granularity benchmark uses O(1) work per booking attempt and O(number of seats) booking-state storage. The counter benchmarks perform O(1) work per update; all N thread updates require O(N) total operations. Synchronisation overhead affects measured time but not these basic operation counts.

Correctness requires the semaphore count and protected buffer state to remain consistent. Every successful producer publication must correspond to one consumer removal. The exact-once check complements the arithmetic sum because duplicate/missing values can sometimes cancel numerically. A data-race-free design is required; a result that happens to match in one unsafe run is not a proof of correctness.

12. Results and Conclusion

The completed bounded-buffer implementation demonstrates how the empty/full semaphores coordinate availability while the mutex serializes updates to circular-buffer indices. The 1P/1C test must produce 20 items with sum 210; the 4P/4C test must process 80 unique items with sum 3,240. The 100-run test verifies repeatability of the original 20-item case.

For railway booking, a single global lock offers simple correctness but can limit throughput. Per-train and per-coach locks allow unrelated booking operations to proceed concurrently. The finest practical lock granularity should be selected from the measured throughput while ensuring that multi-resource operations acquire locks in a consistent order. Peterson's algorithm illustrates two-participant mutual exclusion under explicit memory-order assumptions. Semaphores and monitors provide higher-level coordination for producer-consumer, readers-writers and dining-philosophers problems. For a simple counter, atomic_fetch_add is efficient; spin locks waste CPU when held or contended for too long, while mutexes are more appropriate for longer waits.

13. Execution Procedure and Evidence

1. Place bounded_buffer.c, peterson_demo.c, booking_locks.c, atomic_spin_benchmark.c and run_task5.sh in one Task05 folder.
2. In Ubuntu/WSL, run chmod +x run_task5.sh.
3. Run ./run_task5.sh.
4. Confirm 1P/1C output has sum 210 and exact-once PASS.
5. Confirm 4P/4C output has sum 3,240 and exact-once PASS.
6. Confirm bounded_buffer_100_runs.txt reports 100/100 runs passed.
7. Save booking_lock_results.txt and atomic_spin_results.txt; record environment.txt.
8. Capture screenshots of the output and include the generated files in the Task05 folder.